

stabcodes__repo

Coordinate-free stabilizer-code circuit builder & decoder

Overview

stabcodes__repo builds Stim circuits and detector-error models (DEM) from pasted or file-based stabilizer lists plus a per-stabilizer CNOT layer schedule, then sweeps physical error rates using Sinter + PyMatching and plots the logical error per shot.

Outputs

- `circuit.stim` (Stim circuit)
- `dem.txt` (Detector Error Model, when construction succeeds)
- `build_meta.json` (metadata for decoding sweeps)
- `metrics.{json,csv}, plot.png`

Install

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Quick Start

A) Interactive build

```
python -m stabcodes.cli.build
```

Paste stabilizers (or dense Pauli rows), then enter:

- rounds R
- layers per round L
- per-stabilizer layer numbers for each support index ($1..L$), in order

B) Build from a benchmark file

Example ($d=3$ rotated surface):

```
python -m stabcodes.cli.build \
  --out results/d3_rotated_surface \
  < benchmarks/rotated_surface/d3.txt
```

Decode and plot

```
python -m stabcodes.decode.run \  
  --meta results/<run>/build_meta.json \  
  --out  results/<run> \  
  --shots 200000 \  
  --plist 0.001,0.003,0.005,0.007,0.009
```

Input File Format

Benchmark files are plain text:

```
# stabilizers (one per line), sparse form like:  
Z0 Z1 Z5 Z7  
X1 X2 X4 X5  
... (more lines)  
<blank line>  
R  
L  
# One line per stabilizer: comma-separated layer numbers (1..L),  
# matching the order of indices shown for that stabilizer.  
4,2,3,1  
2,1  
...
```

Notes:

- Stabilizers can be sparse (Z_k/X_k tokens) or dense Pauli strings.
- The blank line separates stabilizers from numeric parameters.
- Each layer-assignment line has exactly as many numbers as the stabilizer weight.

Included Benchmarks

- benchmarks/rotated_surface/d3.txt
- benchmarks/rotated_surface/d5.txt
- benchmarks/reption_code/repetition.txt
- benchmarks/color_code/713_color_code.txt

Reproducing Runs

d=3 rotated surface

```
python -m stabcodes.cli.build \  
  --out results/d3_rotated_surface \  
  < benchmarks/rotated_surface/d3.txt
```

```
python -m stabcodes.decode.run \  
  --meta results/d3_rotated_surface/build_meta.json \  
  --out  results/d3_rotated_surface \  
  --shots 200000 \  
  --plist 0.001,0.003,0.005,0.007,0.009
```

```
--meta results/d3_rotated_surface/build_meta.json \
--out results/d3_rotated_surface \
--shots 200000 \
--plist 0.001,0.003,0.005,0.007,0.009
```

d=5 rotated surface

```
python -m stabcodes.cli.build \
--out results/d5_rotated_surface \
< benchmarks/rotated_surface/d5.txt
```

```
python -m stabcodes.decode.run \
--meta results/d5_rotated_surface/build_meta.json \
--out results/d5_rotated_surface \
--shots 200000 \
--plist 0.001,0.003,0.005,0.007,0.009
```

Repetition code (Z-only)

```
python -m stabcodes.cli.build \
--out results/repetition_code \
< benchmarks/repetition_code/repetition.txt
```

```
python -m stabcodes.decode.run \
--meta results/repetition_code/build_meta.json \
--out results/repetition_code \
--shots 200000 \
--plist 0.001,0.003,0.005,0.007,0.009
```

713 color code (Z-only)

```
python -m stabcodes.cli.build \
--out results/713_color_code \
< benchmarks/color_code/713_color_code.txt
```

```
python -m stabcodes.decode.run \
--meta results/713_color_code/build_meta.json \
--out results/713_color_code \
--shots 200000 \
--plist 0.001,0.003,0.005,0.007,0.009
```

Troubleshooting

expected at least one line of stabilizers Ensure your file contains only stabilizer lines before the blank line (no prompts or > prefixes).

DEM fails: non-deterministic observables For Z-only circuits, the builder uses Z resets/measures with a Z logical; for CSS circuits, provide both X and Z stabilizers and a valid layer schedule.

Decoder fallback If the compiled sampler fails on your platform, the script falls back to Stim sampling + PyMatching and still produces metrics/plots.

Different error rates than a reference Confirm stabilizer content/order, layer assignments, R/L, shot count, and `--plist` match; also check noise knobs if you edited them.

Repo Layout

```
benchmarks/  
  rotated_surface/    d3.txt, d5.txt  
  reption_code/       repetition.txt  
  color_code/         713_color_code.txt  
results/  
  <run>/ circuit.stim, dem.txt, build_meta.json, metrics, plot  
src/stabcodes/  
  cli/build.py        # builder (interactive / file)  
  decode/run.py       # decode + plot  
  io/, model/, schedule/, build/
```

License

MIT. See LICENSE.